DATA ANALYSIS WITH PYTHON

Introduction to Data Analysis

- **What is Data Analysis:**
  - Process of **inspecting, cleansing, transforming**, and **modeling data** to **discover useful information, informing conclusions, and support decision-making**.
- **Data Analysis Process:**
  - **Data Extractions**: Data can come from databases, CSV or Excel files, or web APIs.
  - **Data Cleaning**: Preparing data by handling inconsistencies or missing values.
  - **Data Wrangling**: Rearranging, merging, or reshaping data to fit analysis needs.
  - **Analysis**: Using inferential statistics and visualizations to identify patterns or anomalies.
  - **Action**: Sharing insights with stakeholders in different departments via reports tailored to their needs.
  - The process is **iterative and cyclical** rather than strictly linear.
- **Data Analysis vs Data Science**
  - Data Scientists typically have **stronger programming and math skills** and work on machine learning and ETL pipelines.
  - Data Analysts focus more on **communication and storytelling** through reports.

Data Analysis Example A

- **Data Loading:**

    - Data is loaded into a DataFrame, a specialized data structure in Python (from the pandas library).
    - The **DataFrame represents the CSV content** with additional features such as strict data types per column.
- **Data Understanding:**
    - Analysts use the shape of the DataFrame to know the number of rows and columns without inspecting every row visually.
    - The **.info() method** is used to gather quick metadata about the columns, including:
        - Column names
        - Data types
        - Null value counts
- **Statistical Understanding:**
    - Using the **.describe() method**, numeric columns are summarized to reveal statistical properties such as mean, median, minimum, maximum, and quartiles. Example insights:
        - The average customer age is approximately 35 years.
        - Age ranges from 17 to 87 years.
        - Profit contains negative and very large positive values, suggesting potential outliers or data quality issues that require further cleaning.
- **Focused Column Analysis:**
    - The tutorial drills down on the **unit cost** column:
        - Generates boxplots visualizing distribution, quartiles, median, and outliers.
        - Density plots and histograms are used to visualize the distribution of unit costs.

- **Categorical Data Exploration**:
    - Customer ages are categorized into four groups: seniors, youth, young adults, and adults.
    - The distribution of these groups is analyzed using value counts, bar charts, and pie charts.
    - Adults represent the largest segment in the dataset.
- **Correlation Analysis**:
    - A correlation matrix is created to measure relationships between numeric variables such as:
        - Profit
        - Unit cost
        - Order quantity
        - Revenue
    - Key findings:
        - Strong positive correlation between profit and unit cost, and between profit and unit price.
        - Negative correlation between profit and order quantity, which is counterintuitive and flagged for deeper investigation.
        - Profit also shows a positive correlation with revenue.
    - Correlation is visually represented with colors: **blue indicates high positive correlation**, and **red indicates other correlations**.
- **Further Visualizations:**
    - **Scatterplots** examine relationships between:
        - Customer age and revenue

        - Revenue and profit (showing linear dependency)
    - **Boxplots** are created to analyze profit per age group, highlighting how profit varies with customer demographics.

Data Analysis Example B

- **Reshaping and Deriving Data Columns:**
    - Python allows **creating new columns** from existing data efficiently:
    - Example: Calculating revenue per age by dividing revenue by age.
    - Creating a **calculated cost** column as the product of total order quantity and unit cost.
    - Comparing calculated columns against existing ones to **detect data inconsistencies or errors**.
- **Performance Comparison:**
    - Python operations on large datasets are **instantaneous**, while spreadsheet formulas become slow or impractical.
- **Visualization**
    - Generating both **density plots** and **histograms** for data like revenue per age or total revenue, which helps in quickly understanding distributions.
- **Filtering and Aggregation:**
    - Python simplifies filtering:
        - Extracting sales data by state (e.g., Kentucky).
        - Calculating average revenue by age group and country.
        - Filtering by multiple conditions (age group plus country).
- **Data Exploration and Summary:**

- After loading data from the database:
    - Checking **dataset shape** and **structure** (e.g., 16,000 rows).
    - Using **.describe()** for statistical summaries.
    - Visualizing distributions with box plots and categorical analysis

---

How to use Jupyter Notebooks Intro

- **Jupyter Notebook** is an **open-source ecosystem** widely used for interactive data analysis and Python programming.
- **JupyterLab** is described as the **evolution of Jupyter Notebook**, providing a more sophisticated interface including features like:
    - Tree view of files
    - Git integration
    - File previews and other productivity enhancements
- **Why use Jupyter Notebook?**
    - It is an **interactive, real-time environment** for coding and data exploration.
    - Unlike tools such as Excel or Tableau, Jupyter does **not rely on constantly viewing raw data tables**, which become unmanageable at scale (e.g., 100,000+ rows).
    - The user interacts with data by **querying and processing it programmatically**, maintaining a mental model of data rather than visually scanning it.
    - Jupyter serves as a **fully featured Python interpreter** that responds immediately to code execution.
    - It allows mixing **code cells with markdown cells** to document and format notes, making it ideal for both coding and explanation in one place.
- **Core Concepts of Jupyter Notebook**

- The notebook consists of **multiple cells**, each acting as an independent unit of execution or formatting.
- Cells can be:
  - **Code cells**: Execute Python code (e.g., 1 + 3 returns 4).
  - **Markdown cells**: Render formatted text using Markdown syntax to create titles, bold text, quotes, links, images, etc.
- Markdown supports HTML-like formatting rules:
  - Titles are created with # symbols (e.g., # for the largest title).
  - Links, bold text, and images (denoted by ! before the image URL) are supported for rich documentation.
- Users can **add, delete, and execute cells independently**, facilitating an interactive workflow.

Jupyter Notebooks Cells

- **Cell Types**
  - Jupyter Notebook cells come in two primary types:
    - **Code cells**: Contain executable Python code.
    - **Markdown cells**: Contain formatted text for documentation and reporting.
- **Exporting Notebooks**
  - Completed notebooks can be **exported as PDF or HTML**, making them ideal for sharing and presentation.
- **Execution Order and Numbering**
  - Each code cell execution is assigned a **unique execution number**. This helps to track the order in which cells were run, even if cells are rearranged in the

notebook. It ensures clarity on the sequence of operations despite structural changes.

- **Two Modes of Operation**
  - Jupyter Notebook is a mode-based editor, similar to editors like Vim:
    - **Command Mode**: Cell is not being edited; keyboard shortcuts perform notebook-level commands. The cell content is grayed out.
    - **Edit Mode**: Cell is being edited; keystrokes add or modify cell content. The prompt shows a blinking cursor.
  - **Switching between modes**:
    - **From Edit to Command Mode**: Press the **Escape key**.
    - **From Command to Edit Mode**: Press the **Return (Enter) key** or click inside the cell.
- **Essential Keyboard Shortcuts**
  - A – Create a new cell above the currently selected cell (Command Mode)
  - B – Create a new cell below the currently selected cell (Command Mode)
  - D, D – Press D twice quickly to delete the selected cell (Command Mode)
  - Z – Undo the last cell operation (Command Mode)
  - Ctrl + Return – Execute current cell without moving selection (Edit Mode)
  - Shift + Return – Execute current cell and move to next cell (Edit Mode)
  - M – Change cell type to Markdown (Command Mode)
  - Y – Change cell type to Code (Command Mode)
  - X – Cut selected cell (Command Mode)
  - C – Copy selected cell (Command Mode)
  - V – Paste cell below current selection (Command Mode)

    - **Navigation**: Use arrow keys (up/down) to move between cells.
    - When in **Command Mode**, **typing letters triggers commands** rather than inserting characters.
    - When in **Edit Mode**, **typing inserts text** normally.
- **Important Insights**
    - **Mode awareness** is critical for efficient notebook usage; understanding the difference between Command and Edit modes prevents unintended inputs or commands.
    - Execution numbering provides a **transparent history** of code execution order, which is essential for reproducibility and debugging.

Jupyter Notebooks Importing and Exporting Data

- **Jupyter Notebooks excel in visualization**, allowing users to display graphs and charts inline without additional configuration, unlike traditional Python terminals.
- The **matplotlib library** is a first-class citizen in Jupyter, enabling easy display of static images such as price charts directly in the notebook.
- In addition to matplotlib, the video introduces **Bokeh**, a Python library for **interactive visualizations** that allow zooming, panning, and dynamic interaction with charts, contrasting with static matplotlib images.
- **Static images (matplotlib)** are sometimes **preferable for exporting reports** (e.g., PDFs), whereas **Bokeh's interactivity is more suitable for in-depth data exploration**.
- Jupyter Lab handles various file formats including CSV, Excel, XML, and JSON with built-in editors and tree views, enhancing data management.
- Users can easily export analyzed data to Excel spreadsheets directly from Jupyter, facilitating further use and sharing.

Numpy Introduction A

- **Key Insights About NumPy:**
  - **NumPy** is one of the oldest and **most important numeric processing libraries** in Python.
  - Its primary purpose is **numeric computation** and **efficient processing of large numerical datasets**.
  - NumPy has a **limited and focused scope**: it's designed only for numeric processing, offering a simple yet consistent API.
  - **Pure Python** is inherently **slower for numeric computations**, especially with large data, due to Python's high-level nature.
  - NumPy provides a **bridge to high-performance computations** by leveraging optimized low-level implementations, enabling faster execution while maintaining a Pythonic API.
  - Although NumPy is essential, users often interact indirectly with it through libraries like pandas and matplotlib, which rely on NumPy for numeric operations.
- **Core Concepts: Binary Representation of Numbers**
  - Computers fundamentally operate on **binary data — ones and zeros** — since hardware circuits work with two states.
  - **Data is stored in RAM (Random Access Memory)** for active processing, while long-term data resides on hard drives.
  - The size of memory (e.g., 8 GB) translates to a large number of bits available to store data.
  - Understanding how numbers are stored in binary is key to **optimizing memory usage and computational speed**.
- **Binary Representation and Bit-Byte Relationship**
  - Decimal numbers must be converted to binary for storage and processing.

    - Each binary digit (bit) represents an increasing power of 2.
    - The number of bits needed grows with the size of the number
    - For n bits, the total number of decimal values that can be represented is:
        - Total decimal numbers = 2^n

- **Practical Implications for NumPy and Data Processing**
    - Understanding binary representation enables **efficient memory allocation** by choosing appropriate data types (e.g., 8-bit integers vs. 64-bit integers).
    - NumPy leverages these low-level efficiencies to **reduce memory footprint and improve computational speed**.
    - This is particularly important when processing large datasets or when working within hardware constraints.
    - By optimizing bit usage and numeric types, **NumPy helps squeeze maximum performance out of CPU and memory resources**.

- **Conclusion**
    - **NumPy's importance lies in its ability to provide a simple Python interface to highly optimized numeric computations**, crucial for performance-demanding data processing tasks.
    - The underlying efficiency comes from careful management of binary data representations, memory usage, and numeric precision.
    - A clear understanding of **binary number storage and bit allocation is fundamental** to appreciating why libraries like NumPy exist and how they achieve performance gains.
    - The tutorial encourages learners to grasp these low-level concepts but also reassures that practical usage of NumPy does not require deep understanding of these details.

Numpy Introduction B

- **Data Types and Value Ranges:**
  - Different columns in a dataset (even if numeric) can have vastly different ranges and implications for memory storage.
- **Memory Efficiency in Python vs. NumPy:**
  - A simple integer in pure Python consumes much more memory than theoretically necessary (around **20 bytes**, significantly larger than the minimal bits required).
  - This overhead is due to Python's design as a high-level, object-oriented language where numbers are stored as objects with additional attributes, trading memory for ease of use and flexibility.
  - NumPy allows precise control over the number of bits used to store integers (e.g., 8-bit, 16-bit, 32-bit, 64-bit), enabling much more memory-efficient storage.
- **NumPy's Array Storage and Performance Advantages:**
  - Unlike Python lists, NumPy arrays store elements in **contiguous memory locations**, which improves cache locality and access speed.
  - This contiguous storage allows NumPy to leverage **advanced CPU instructions** for fast numeric and matrix operations.
  - NumPy arrays are optimized for numerical computing and can use specific bit-width data types to reduce memory usage and improve computational speed.
- **Scalability and Optimization:**
  - For small datasets (thousands of records), memory optimization is less critical, but for very large datasets (billions or trillions of records), choosing the smallest adequate bit size can lead to significant memory and computational savings.

Numpy Arrays

- NumPy is a lightweight but powerful library focused on arrays and numeric data types.
- It provides simple, high-performance alternatives to Python lists for large data volumes.
- **Creating NumPy Arrays:**
    - Arrays are created using the syntax.
    - Arrays resemble Python lists but differ significantly in performance and memory layout.
    - Arrays support zero-based indexing, slicing, negative indexing, and steps, just like Python lists.
- **Advanced Indexing:**
    - NumPy **supports multi indexing**, allowing selection of multiple elements simultaneously via a list of indices.
- **Data Types in NumPy:**
    - Arrays have fixed, continuous data types for memory efficiency and speed.
    - Default data types depend on the platform (64-bit platform defaults to 64-bit integers/floats).
    - NumPy **primarily handles numeric types** (integers, floats, Booleans) but **can store strings encoded in Unicode**.
    - Storing arbitrary Python objects is unsupported or inefficient in NumPy.
- **Multi-Dimensional Arrays:**
    - NumPy supports arrays of arbitrary dimensionality:
        - 1D arrays (vectors)
        - 2D arrays (matrices)

        - 3D arrays (tensors/cubes)
    - Important attributes include
        - shape – Dimensions of the array
        - ndim – Number of dimensions
        - size – Total number of elements
    - When dimensions mismatch during creation, NumPy defaults to an array of type object and reduces dimensionality.
- **Indexing and Slicing Multi-Dimensional Arrays:**
    - Indexing accesses rows first, then columns
    - Multi-dimensional indexing uses comma-separated indices.
    - Slicing syntax allows selecting ranges per dimension
    - Negative indexing and step values work similarly to Python lists.
- **Modifying Arrays:**
    - Entire rows or slices can be reassigned new values if dimensions match.
    - NumPy supports broadcasting to expand single values across slices, e.g., assigning a scalar to a row.
    - Built-in Operations and Aggregations:
    - NumPy arrays support many vectorized operations for statistical calculations, such as:
        - Sum (∑)
        - Mean (average)
        - Standard deviation (σ)
        - Variance

- These operations can be applied to whole arrays or along specific axes (rows or columns).
- Axis parameter:
  - axis=0: operate across rows (column-wise)
  - axis=1: operate across columns (row-wise)
- Works seamlessly with higher-dimensional arrays by increasing the axis index.

Numpy Operations

- **Vectorized Operations**: These are operations performed element-wise between arrays or between an array and a scalar. Instead of looping explicitly over each element, operations like addition, multiplication, etc., are applied simultaneously across the entire array.
- **Broadcasting**: This refers to how NumPy applies operations between arrays of different shapes by automatically expanding the smaller array to match the larger one without copying data. This automatic expansion enables element-wise operations on arrays that aren't exactly the same shape but are compatible in dimension.
- **Immutability of Arrays**: Most NumPy operations do not modify the original array but instead return a new array with the result of the operation. This behavior can be overridden by using in-place operators like +=, -=, *=, etc.
- **Key Insights:**
  - **Vectorized operations are fundamental to NumPy's performance advantages** because they avoid explicit Python loops and utilize optimized C-based implementations.
  - **Broadcasting extends the flexibility of operations** to arrays of different shapes, enabling more expressive and concise code.
  - **NumPy arrays are generally immutable in operations**, meaning the original data is preserved unless explicitly modified with in-place operators.

- Understanding broadcasting and vectorized operations is critical for efficient numerical computing and is also widely used in libraries like pandas with similar syntax and primitives.

Numpy Boolean Arrays

- **Boolean arrays function similarly to vectorized arithmetic operations**; they operate element-wise on arrays but return Boolean values (True or False) instead of numeric results.
- Boolean operations allow for **dynamic, condition-based filtering** of array elements without manually specifying each index.
- There are multiple ways to select elements in an array:
  - **Using traditional indexing**
  - **Multi-index selection**
  - **Boolean array indexing**, where a Boolean array of the same length is passed to select elements corresponding to True values.
- **Boolean indexing:**
  - An array of Boolean values is used to select elements from another array. The length of the Boolean array must match the target array, and only elements corresponding to True are selected.
- **Broadcasting Boolean operations:**
  - Applying a condition such as a≥2 on an array a=[0,1,2,3] produces a Boolean array [False,False,True,True]. This result can be used to filter 'a' directly.
- **Combining Boolean conditions:**
  - Boolean arrays can be combined with logical operators like OR (pipe symbol |) and AND (ampersand &) to create complex filters.

- **Efficiency and scalability:**
    - Manually writing Boolean arrays for large datasets (e.g., millions of elements) is impractical, but these Boolean arrays are **usually generated dynamically through vectorized conditions**, enabling efficient filtering without loops.

Numpy Algebra and Size

- **NumPy's Linear Algebra Efficiency:**
    - NumPy has optimized implementations of core linear algebra operations such as dot products, cross products, and matrix transpositions. These functions are implemented at a low level, making them extremely fast and essential for applications like **machine learning.**
- **Memory Efficiency Comparison (Python vs. NumPy):**
    - A **standard integer** in Python consumes **28 bytes** of memory, which is significantly large for storing simple numbers.
    - NumPy allows control over the number of bytes per integer making it much more **memory-efficient**.
    - This fine-grained control over data size is critical for handling large datasets efficiently.
- **Performance Comparison for Array Operations:**
    - NumPy's equivalent operation was significantly faster, staying within the microsecond range even when adding more elements.
    - When the array size increases, Python's execution time moves into the millisecond range, while NumPy remains much faster.
    - This illustrates NumPy's superior scalability and performance for numerical computations.

Pandas Introduction

- **Pandas as a Core Library:**
  - It supports **data import from various sources** such as databases, Excel files, and CSV files.
  - Enables **data processing tasks** including combining, merging, and statistical analysis.
  - Facilitates **data visualization** (e.g., bar charts) and **report creation.**
  - Recently released version 1.0, highlighting its maturity and extensive use in the Python ecosystem.
- **Primary Data Structures in Pandas:**
  - The two fundamental structures are:
    - Series
    - DataFrame (similar in appearance to Excel tables, more commonly used)
- **Definition and Nature:**
  - A Series is an **ordered sequence of elements**.
  - Each element is associated with an **index**, making it similar to both lists and dictionaries.
  - Internally, a Series is backed by a **NumPy array**.
  - Unlike Python lists, Series have a **fixed data type**, inherited from NumPy's array constraints.
- **Structure and Indexing:**
  - A Series has an **associated name**, which becomes especially meaningful when it forms part of a DataFrame column.
  - Indexing in Series is **explicit and label-based**, unlike Python lists where indexing is positional.

    - The **default index is a range index** starting from 0 to n − 1 (number of elements minus one).
    - Users can **arbitrarily assign custom indices** to a Series, enabling access to elements by meaningful labels rather than numeric positions.
    - This custom indexing makes a Series behave somewhat like a **dictionary with ordered keys**, but with the key distinction that Series are **ordered data structures**.
- **Comparison with Python Data Structures:**
    - Series share properties with lists (ordered sequences) and dictionaries (key-value pairs).
    - However, it is unique in that it is **both ordered and indexed by arbitrary labels**.

Pandas Indexing and Conditional Selection

- **Pandas Series** is an ordered sequence of elements backed by a **NumPy array**, offering **high efficiency and speed**.
- Each element in a Series is associated with an **index label**, which can be any hashable type (e.g., strings like country names).
- Series supports:
    - **Label-based indexing** (similar to Python dictionaries)
    - **Position-based indexing** using .iloc (integer-location based indexing)
    - **Multiple indices and slicing**
- **Boolean Series and Conditional Filtering**
    - Boolean operations on Series allow **efficient filtering and querying akin** to NumPy Boolean arrays.

    - Example: Filtering countries with population greater than a threshold produces a Boolean Series.
    - Boolean operators such as **& (and), | (or), and ~ (not)** are supported for combining conditions.
    - Statistical functions like **mean** and **standard deviation** can be used in conjunction with Boolean selections to create complex filters.
- **Vectorized Operations and Broadcasting**
    - Mathematical operations on Series are **vectorized**, allowing element-wise operations without explicit loops.
    - Boolean operations act similarly to arithmetic operations in terms of vectorized execution.
- **Modifying Series Data**
    - Values in a Series can be modified using:
        - **Label-based indexing**: e.g., setting the population of "Canada" to a new value.
        - **Position-based indexing**: e.g., modifying the last element using .iloc.
        - **Boolean indexing**: e.g., setting all countries with population less than 70 million to a specific value.
    - These capabilities facilitate data cleaning and transformation.

Pandas DataFrames

- **Data Frame Definition:**
    - A data frame is analogous to an Excel table, consisting of rows and columns. It can be thought of as a collection of multiple series, where each column in the data frame corresponds to a **series**.

- **Data Frame Structure:**
    - **Columns** have names and associated data types (e.g., integer, float, object/string).
    - **Rows** are indexed, which enables efficient data selection.
    - **Data frames** can be easily created from CSV files or other tabular data formats.
- **Key Attributes and Methods of Data Frames**
    - .columns – Lists the column names of the data frame.
    - .index – Shows the index labels of the rows.
    - .info() – Provides a summary of the data frame: columns, data types, and number of non-null entries per column (important for data cleaning).
    - .shape – Returns a tuple representing the dimensions of the data frame (rows, columns).
    - .describe() – Generates summary statistics for numeric columns: count, mean, std deviation, min, max, and percentiles (25th, 50th, 75th).
- **Data Types in Data Frames**
    - Columns have inherent types, and Pandas automatically infers them:
        - **Object**: Usually strings (e.g., continent names).
        - **Float**: Numerical columns with decimal values (e.g., HDI).
        - **Integer**: Whole numbers (e.g., surface area).
- **Data Selection in Data Frames**
    - .loc[] – Select by index label
    - .iloc[] – Select by positional index
    - df['column'] or df.column – Select entire column

- **Key Insights**
    - A **data frame is a two-dimensional** labeled data structure with columns as series.
    - The **.info()** and **.describe()** methods are essential for **exploring data frame structure and summary statistics**, particularly useful in data cleaning and analysis.
    - Data frames support **multiple ways to index and select data: .loc for label-based, .iloc for positional,** and bracket notation for columns.
    - The selections from a data frame typically return series, maintaining consistency in data representation.

Pandas Conditional Selection and Modifying DataFrames

- **Conditional Selection using Boolean Arrays/Series**
    - Filtering data frames based on conditions, this involves creating a Boolean series where each element corresponds to whether a row meets the condition (e.g., True or False).
    - The Boolean series is then used with .loc to filter the data frame rows.
    - The critical mechanism is **index alignment**: the Boolean series and the data frame must share the same index for filtering to work correctly.
- **Selecting Rows and Columns Simultaneously**
    - After selecting rows with Boolean indexing, one can specify columns to retrieve
    - The syntax supports multi-dimensional filtering: **.loc[rows_condition, columns]**
- **Dropping Rows or Columns**
    - Rows or columns can be dropped by specifying their labels.

    - Examples include **df.drop('Canada')** to drop a row or **df.drop(['Population', 'HDI'], axis=1)** to drop columns.
    - Dropping can be done by index or by column names.
    - There are more advanced access methods similar to NumPy, but these are not recommended for beginners.
- **Broadcasting Operations Between Series and DataFrame**
    - Broadcasting allows operations like subtraction to be applied between a series and a data frame.
    - Alignment occurs based on column names, ensuring correct operation application across matching columns.
    - This demonstrates pandas' ability to **align independent objects by their labels during arithmetic operations**.
- **Immutability of Pandas Operations**
    - Most pandas operations do not modify the original data frame but return a new object with the applied changes.
    - For instance, df.drop('Canada') returns a new data frame excluding Canada, but df remains unchanged unless reassigned.
    - This behavior mirrors NumPy's immutability in many operations.
    - To persist changes, one must explicitly reassign.
- **Adding and Modifying Columns**
    - New columns can be added by assigning a pandas series to a new column name, **df['language'] = language_series**
    - The series does not need to cover all indices; pandas matches based on index labels and inserts NaN where data is missing.
    - Modifying an existing column is as simple as assigning a new value or series to that column, **df['language'] = 'English'** sets all values to English.

- **Detecting Operations That Modify the DataFrame**
    - Operations with an assignment (=) or in-place operators (e.g., +=) typically modify the original data frame.
    - Functions like .rename() do not modify by default but return a new data frame unless inplace=True is specified (not shown in this video).
    - Attempting to rename non-existent labels does not cause errors; pandas ignores them gracefully.

Pandas Creating Columns

- **Creating New Columns by Combining Existing Ones**
    - Example: Computing **GDP per capita** by dividing the GDP column by the population column.
    - This operation returns a **pandas Series**, which can be assigned as a new column to the existing DataFrame.
    - These arithmetic operations are highly efficient as they leverage **NumPy array broadcasting**.
- **Understanding pandas Data Structures**
    - The **DataFrame is a 2D** tabular data structure with rows and columns.
    - The **Series is a 1D l**abeled array representing a single column or row.
    - Many operations on DataFrames result in Series, which can be reassigned or manipulated independently.
    - Grasping the distinction between DataFrame and Series is essential for effective data manipulation.

- **Reading External Data with pandas**
    - The **read_csv()** function is introduced as a primary method to import CSV files into a pandas DataFrame.
    - Other similar functions include read_sql, read_excel, read_xml, and read_html, allowing import from various formats.
    - Default behavior of read_csv() considers the first row as column headers.
    - In cases where the CSV lacks headers, the parameter **header=None** disables automatic header inference, requiring manual column naming.
- **Data Inspection and Cleaning**
    - Methods like **df.head()** and **df.tail()** allow viewing the first and last few rows of large datasets.
    - **df.shape r**eturns the dimensions of the DataFrame (e.g., 365 rows, 2 columns).
    - **df.info()** provides column data types and non-null counts.
    - Initial import treated the timestamp column as an object (string), which requires conversion to proper datetime format using pd.to_datetime().
    - Once converted, the timestamp column can be set as the index of the DataFrame, enabling intuitive and fast access to data by date.
- **Optimizing Data Import**
    - The video demonstrates how to combine multiple preprocessing steps into a single **read_csv()** call with appropriate parameters:
        - **header=None** to disable header inference.
        - **names=[...]** to assign column names explicitly.
        - **index_col=0** to set the first column as the index.
        - **parse_dates=[0]** to parse the index column as datetime.
    - This approach simplifies and automates data ingestion workflows.

- **Basic Plotting with pandas**
    - The **built-in df.plot()** method is a simple interface to create visualizations using the **matplotlib library.**

Data Cleaning Introductions

- **Four-Step Conceptual Process of Data Cleaning:**
    - **Identifying Missing Data**
        - Missing entries such as missing car names or prices in a dataset.
        - Strategies include dropping records with missing values or filling them (e.g., filling missing prices with the average price)
        - Missing data issues may require communication with data providers or ETL teams, especially when data is purchased or critical.
    - **Handling Invalid Values**
        - Example: Non-numeric strings in numeric columns (e.g., price column containing strings).
        - Easier to detect and fix than domain-specific errors.
    - **Domain-Specific Validation**
        - Values that are technically valid but unrealistic in context (e.g., a customer age of 170 years)
        - Requires domain knowledge to identify such anomalies.
        - More complex since domain expertise or external validation is needed.
    - **Advanced Domain Challenges**
        - In unfamiliar domains (e.g., biology), determining valid ranges or values requires domain expert input.

        - This represents the highest complexity in data cleaning.

- **Pandas and NumPy Integration for Missing Data Identification**
    - **Dependence on NumPy's missing value system:**
        - Uses special markers like **NaN (Not a Number)** and **None** for missing data.
    - **Core Pandas Functions for Missing Data:**
        - **isna() and isnull()** — complete synonyms, return Boolean masks indicating missing data.
        - **notna() and notnull()** — also synonyms, return opposite Boolean masks indicating non-missing data.
    - **Usage Notes:**
        - These functions can be applied to individual values, entire Series, or DataFrames.
        - Boolean results can be summed to count missing or non-missing values, leveraging Python's treatment of True as 1 and False as 0.
    - **Examples of Boolean Operations:**
        - Summing isna() over a Series counts how many missing values exist.
        - Summing notna() counts how many valid (non-missing) values exist.

- **Important Usage Details**
    - **pd.isna()** – Detects missing values (NaN, None)
    - **pd.notna()** – Detects non-missing values
    - **.dropna()** – Drops missing values from Series/DataFrame
    - These methods are **immutable**: they do not modify the original data structures but return new objects with the applied transformations.

Data Cleaning with DataFrames

- **Inspecting DataFrames:**
  - Use **info()** to get an overview of the DataFrame, including the total number of rows and count of non-null values per column.
  - **shape** reveals the structure (rows and columns).
- **Identifying Missing Data:**
  - **df.isna().sum()** or **df.isnull().sum()** quickly counts the number of missing values per column.
  - Boolean tests like **any()** and **all()** help verify if any or all values meet a condition
- **Dropping Missing Values:**
  - **dropna()** removes rows or columns containing missing values.
    - Default behavior: Drops any row with at least one null value.
  - **Axis parameter**:
    - axis=0 (default): Drop rows.
    - axis=1: Drop columns.
  - **Threshold parameter** allows controlling how many non-null values must be present to keep a row/column.
- **Filling Missing Values (Imputation):**
  - **fillna(value)** replaces missing values with a specified constant (e.g., zero).
  - **fillna(method='ffill') (forward fill)**: Propagates last valid observation forward.
  - **fillna(method='bfill') (backward fill)**: Propagates next valid observation backward.
  - Axis parameter controls direction of fill:

        - axis=0: Vertical fill (column-wise).
        - axis=1: Horizontal fill (row-wise).
    - **Imputation methods** do not alter the original data but return a new series or DataFrame.
- **Handling Invalid Data (Non-null but Incorrect):**
    - Some columns, like categorical ones, have constrained valid values.
    - Example: A sex column allows only values {‘m’, ‘f’, ‘d’, ‘?’}, where ‘n’ or other values are invalid.
    - Use **value_counts()** or **unique()** to identify invalid categories.
    - Cleaning invalid values can be done with replace() to map incorrect values to valid ones.
- **Complex Numeric Errors:**
    - Example: Age column showing values like 290, 320, which are invalid human ages.
    - Requires domain knowledge and more advanced fixes (e.g., correcting by dividing values by 10 to fix typos)
    - Such corrections often need custom coding beyond simple pandas functions.

Data Cleaning Duplicates

- **duplicated() Method:**
    - By default, it marks all but the first occurrence of a duplicate as True (duplicates).
    - It works top-down, checking if the current value has appeared before.
    - The method accepts a keep parameter to specify which duplicates to mark:

        - **keep='first'** (default): First occurrence is not a duplicate; subsequent ones are duplicates.
        - **keep='last'**: Last occurrence is not a duplicate; previous ones are duplicates.
        - **keep=False**: Marks all duplicates as True.
    - Example: For repeated country values like "UK" or "Germany," the function can identify which rows are duplicates based on the chosen rule.
- **drop_duplicates() Method**:
    - Works similarly but removes duplicate rows from a DataFrame or Series.
    - Accepts the same keep parameter to decide which duplicates to retain.
    - Can operate on subsets of columns to define duplicates more narrowly.
- **Subset Parameter for Duplicate Identification:**
    - In datasets with multiple columns, duplicates may be defined based on one or several columns.
    - Example: A player named "Coby" could appear multiple times with different positions ("SG" vs. "SF").
    - Defining duplicates on a subset of columns helps distinguish these cases:
        - Checking duplicates for just the player name alone may mark multiple rows as duplicates.
        - Including position as part of the subset can treat some rows as unique.

Data Cleaning and Visualizations

- **Data cleaning benefits significantly from data visualization**, which helps identify outliers and understand the statistical properties of data.
- **Matplotlib** is the core library behind pandas plotting functions.

- There are **two main Matplotlib APIs**:
    - **The Global API**, which is function-based and inspired by MATLAB, is widely used in tutorials but has drawbacks.
    - **The Object-Oriented (OO) API**, preferred by the presenter, provides clearer, more manageable, and explicit control over plots, especially with multiple subplots.
- The **Global API** modifies a global figure and axes implicitly, making debugging difficult when multiple plots are involved.
- The **OOP API** involves explicitly creating figure and axes objects, then calling plotting methods on these objects, improving readability and maintainability.
- Matplotlib supports various plot types essential for data analysis and cleaning, including:
    - Line plots
    - Scatter plots (including multi-dimensional encoding via color and size)
    - Histograms
    - Kernel Density Estimators (KDE)
    - Bar plots (including stacked bars)
    - Box plots
- **Visualizations can encode multiple data dimensions** simultaneously (e.g., scatter plots encoding x, y, color, and size)
- **Statistical context** is critical when deciding if a data point is an outlier or valid. For instance, a 90-year-old person is a valid outlier in general but invalid in a dataset of high school students.

Reading Data Introduction

- **CSV and Text Files:**
  - CSV (Comma-Separated Values) files are essentially plain text files representing tabular data.
  - The separator in CSV files is typically a comma, but it can vary (e.g., pipes, spaces, or in this case, a greater-than sign $>$).
  - Both CSV and text files are read with similar methods but may require special handling depending on their delimiters and data formatting.
- **Reading Files Using Pure Python:**
  - Before using pandas, understanding how Python reads files at a lower level is beneficial.
  - The built-in Python open function, often used within a **context manager** (with statement), creates a file pointer for safe and efficient file manipulation.
  - **File pointers** can be iterated line-by-line, or read in chunks, which is foundational to processing text data.
- **Challenges in File Parsing:**
  - Different files may use various delimiters, such as commas, colons, pipes, or spaces, complicating the parsing process.
  - Some files may have quoting rules that affect how data is read and interpreted.
  - Parsing such files manually can be error-prone and difficult without specialized tools.
- **Using the CSV Module and pandas:**
  - Python's built-in csv module can handle various delimiters by specifying a delimiter parameter to accommodate non-standard separators like $>$.

- However, for more complex parsing needs and easier data manipulation, pandas offers a high-level abstraction that simplifies reading and processing these files.
- Pandas abstracts many of the underlying challenges, such as handling different delimiters and quote characters, making it the preferred tool for data scientists and analysts.

Reading Data CSV and TXT

- **Pandas' read_* methods:**
  - **read_sql** for SQL databases
  - **read_csv** for CSV files
  - **read_html** for scraping tables from web pages
  - **read_json** for JSON files
- **CSV File Handling in Pandas:**
  - The **read_csv** method is highly customizable with numerous parameters to handle:
    - File paths (local or remote URLs)
    - Presence or absence of headers
    - Different delimiters/separators (e.g., commas, greater signs)
    - String quoting and escape characters
    - Handling of blank lines and missing values (e.g., converting "?", "!", or "-" to NaN)
    - Custom column names and data types with parameters like names and dtype
    - Date parsing with parse_dates

        - Skipping specific rows or blank lines at the start using skiprows or skip_blank_lines

- **Writing Data with to_* Methods:**
    - For every read_* method, there is generally a corresponding to_* method for exporting data:
        - For example, after reading CSV data, the to_csv method allows saving DataFrames to CSV files with customizable options such as delimiter, header inclusion, and index handling

- **read_csv Key Parameters:**
    - **filepath_or_buffer** – Path or URL of the CSV file
    - **header** – Row number(s) to use as the column names
    - **sep / delimiter** – Character that separates fields
    - **na_values** – Additional strings to recognize as NaN
    - **names** – List of column names to use
    - **dtype** – Data types for columns
    - **parse_dates** – Columns to parse as datetime
    - **skiprows** – Rows to skip at the beginning
    - **skip_blank_lines** – Whether to ignore blank lines

Reading Data from Databases

- **Database Libraries and API Consistency:**
    - Depending on the database engine (Postgres, MySQL, Oracle, etc.), different Python libraries are required. However, many libraries conform to Python's DB-API, making the interface to interact with databases uniform across different systems.

- **Manual Query Execution (Pre-pandas Method):**
    - Importing appropriate database modules (e.g., sqlite3)
    - Creating a connection and cursor
    - Executing SQL queries manually (e.g., SELECT * FROM employees LIMIT 5)
    - Fetching results as lists of tuples, which are less immediately useful without further processing.
- **Using pandas to Read SQL Data:**
    - pandas introduced methods such as read_sql() to simplify database reads:
        - **read_sql()** takes a SQL query string and a connection object, returning a DataFrame including column names.
        - Options allow specifying an index column and parsing data types for columns, streamlining data import workflows.
- **Underlying Methods in pandas:**
    - read_sql() is a wrapper for two methods:
        - **read_sql_query()** for executing SQL queries
        - **read_sql_table()** for reading entire tables by name (requires SQLAlchemy for connection setup).
- **Writing Data Back to Databases with pandas:**
    - pandas supports writing DataFrames to SQL tables using to_sql(). Key parameters include:
        - Table name
        - Connection object
        - Behavior if the table exists (fail by default, with options replace or append)

- **Reading Data from HTML Tables**
    - **pandas read_html() Method:**
        - This method can read tables directly from HTML pages or strings, parsing all tables found on a page into a list of DataFrames.
    - **Use Cases and Limitations:**
        - Works well with well-structured HTML sites, such as Wikipedia or statistics portals.
        - Can provide a quick hack to retrieve data like country GDP without downloading datasets.
        - Effectiveness depends heavily on the page's HTML structure; poorly structured tables may require extensive cleaning.
    - **Parsing Process Details:**
        - read_html() downloads the page content if given a URL or accepts raw HTML content.
        - It detects multiple tables and returns them as multiple DataFrames.
        - If tables have headers (<thead>), pandas recognizes those automatically.
        - If headers are missing (header row is just another row), additional parameters or manual cleaning are needed.

Parsing HTML and Saving Data

- **Data source legality and format considerations**: Always verify if the data you intend to parse from websites is **publicly available** and **legally permissible for use**.
- **Parsing HTML tables:**
    - Extracting tables from websites (e.g., Wikipedia) is feasible using pandas.read_html().

    - Complex tables often involve multiple headers, column spans, and row spans, making direct parsing result in messy data frames.
    - HTML tables are optimized for human readability, not machine parsing, leading to issues such as repeated header rows embedded within data.
    - Data cleaning is crucial, including tasks like dropping extraneous header rows that appear periodically in the parsed data frame.
- **Reading Excel files:**
    - Excel files differ from text files, requiring specialized libraries for parsing.
    - **pandas.read_excel()** is the standard method for reading Excel sheets into data frames.
    - Excel files may contain multiple sheets; by default, read_excel reads the first sheet unless specified.
    - Pandas offers an **ExcelFile class** to first load the file and inspect available sheet names before parsing individual sheets.
    - **Parameters allow customization** such as specifying which sheet to read and controlling header and index parsing.
- **Writing to Excel:**
    - Using **pandas.ExcelWriter**, users can write multiple data frames to different sheets within the same Excel file.
    - **Writing is customizable**, including whether to write an index or not.
    - Platform and dependency considerations:
        - Excel reading/writing **depends on external libraries** (e.g., openpyxl, xlrd) which may vary based on operating system (Windows, macOS, Linux).
        - Users should consult pandas documentation for platform-specific instructions and compatibility.

Python Introduction

- **Python Origins and Popularity**
  - Created in the early 1990s by Guido van Rossum.
  - Older than Java, but has gained significant popularity in the last 5–10 years.
  - Guido van Rossum remains an influential figure in Python's development and feature decisions.
- **Python Language Characteristics**
  - High-level, interpreted, and dynamic programming language.
  - Supports **multiple paradigms**: object-oriented, functional (functions as first-class objects), and imperative programming.
  - Known for its **clean syntax** and **indentation-based block structure**.
  - Widely used in **web development**, **scripting**, **system configuration**, and **data science**.
- **Syntax and Code Structure**
  - Functions are defined using the **def** keyword followed by the function name and parameters.
  - **Indentation (usually 4 spaces) replaces curly braces** to define blocks (functions, if-else, loops).
  - **Indentation is mandatory** and improves readability, avoiding common errors related to misplaced braces.
  - Comments start with the # symbol.
- **Variables and Typing**
  - Python has **dynamic and strong typing**.
  - Variables are declared by assignment without explicit type definitions.

    - Python variables can be reassigned to different types as needed.
    - Collections (lists, tuples, dictionaries) are **heterogeneous** (can hold mixed data types).
- **Core Data Types in Python**
    - **int** – Integer numbers
    - **float** – Floating-point numbers; subject to typical floating-point precision issues
    - **decimal** – Provides exact decimal representation to avoid float precision errors
    - **str** – String type, representing Unicode text
    - **bool** – Boolean type with two values: True and False
    - **NoneType** – Represents the absence of a value (None)
- **Python String Handling**
    - Strings are Unicode by default in Python 3.
    - Multi-line strings can be created using triple quotes (''' or """).
    - Python 3 resolved previous distinctions between Unicode and byte strings that existed in Python 2.
    - There is still a conceptual difference between text strings and binary strings, but for most beginners this can be ignored.

Python Functions and Collections

- **Functions in Python**
    - Functions are defined using the **def keyword**, not function.
    - Function parameters are optional.
    - It is recommended to always include a return statement, as functions typically return a value.

    - If no explicit return is provided, Python implicitly returns None.
    - Python supports advanced parameter handling:
        - **Variable-length arguments** (allowing dynamic number of inputs).
        - **Keyword arguments** (named parameters for clarity).
- **Operators and Type Checking**
    - Python includes standard arithmetic operators like addition, modulus, power, etc.
    - Boolean operators include relational operators (>, <, >=, <=) and logical operators (and, or, not).
    - Python is **dynamically typed but strongly typed**, meaning:
        - Types are checked at runtime.
        - Operations between incompatible types raise errors (e.g., comparing integer to string results in a TypeError).
- **Control Flow**
    - Python uses **indentation** to define code blocks.
    - Conditional statements include if, elif, and else.
    - Python **does not have a switch statement**.
    - Looping uses:
        - **for loops** that iterate over collections (lists, ranges, etc.).
        - **while loops** exist but are generally discouraged in favor of for loops to avoid infinite loops.
- **Core Data Structures in Python**
    - **Lists** are versatile and heavily used but mixing many data types inside lists is discouraged for code clarity.
    - Lists are **zero-indexed**; negative indices access elements from the end.

    - **Tuples** are like lists but **immutable**; they cannot be changed after creation.
    - **Dictionaries** provide **named keys** rather than positional indexing, which simplifies access to data fields (e.g., user data with keys like "email", "age").
    - **Sets** are unordered and only contain **unique values**.
- **Important Best Practices Highlighted**
    - Always use **return** in functions to **explicitly return values**.
    - Prefer **for loops** over while loops for iteration **to prevent infinite loops**.
    - **Avoid mixing different data types** within collections for better maintainability.
    - **Use dictionaries** when the dataset **requires label-based access** instead of positional indexing.
    - **Use sets for efficient membership checking** and to automatically handle duplicate entries.

Python Iterations and Modules

- **Iteration in Python:**
    - Python uses a **for loop** to iterate over collections such as lists and dictionaries.
    - Iterating over a dictionary defaults to keys; to access values, use .values().
    - To access both keys and values simultaneously, use .items().
    - Unlike languages with traditional for loops (e.g., for i = 0; i < 10; i++), **Python's for loop iterates over iterable objects directly**.
- **Python's Standard Library and Modules:**
    - Python includes a **vast standard library** maintained by the core Python development team, which is built-in and does not require third-party installation.
    - This library covers diverse domains such as:

        - Security and cryptography
        - Mathematics and numeric computing (e.g., NumPy)
        - Machine learning
        - Web development
        - Game development (e.g., Pygame)
        - Graphical user interfaces
    - Modules are imported to leverage this functionality; there is a distinction between **modules** (single files) and **packages** (collections of modules).
- **Exception Handling:**
    - Errors, such as type mismatches (e.g., comparing a string to an integer), cause exceptions that can terminate a program.
    - Python uses **try-except blocks** to catch and handle exceptions gracefully without crashing.
    - This enables programmers to anticipate and manage errors explicitly.